Introduction to Object-Oriented Software Construction

Experiment Guidebook

Experiment 4: Audio Control and Multithreading

Harbin Institute of Technology (Shenzhen)

2025

1. Objective

(1) Understand the lifecycle and basic concepts of threads in Java
(2) Master two thread implementation methods (Runnable interface, Thread class inheritance)
(3) Use multithreading to manage sound effects and switch between firepower supply states

2. Environment

- Operating System: Windows 10
- IDE: IntelliJ IDEA 2025.2
- JDK: OpenJDK 24 or higher

3. Experiment Tasks

The **goals of this development iteration** are twofold: controlling game sound effects and accurately managing the active duration of firepower supply states.

(1) Sound Effect Requirements

During gameplay, the **background music** should loop continuously and stop as soon as the game ends. Appropriate sound effects must be played when a **bullet hits an enemy aircraft**, when a **power-up supply takes effect**, and when the **game ends**. When the Boss enemy appears, its dedicated background music should start looping immediately. This **Boss music** must stop either when the Boss is destroyed or when the game ends.

(2) Firepower Supply State Timing Requirement

When a **firepower supply** is activated, the hero aircraft should switch to its enhanced firing trajectory and maintain this state for exactly **5 seconds**, after which it automatically **reverts** to the default **straight-shot** trajectory.

4. Experiment Steps

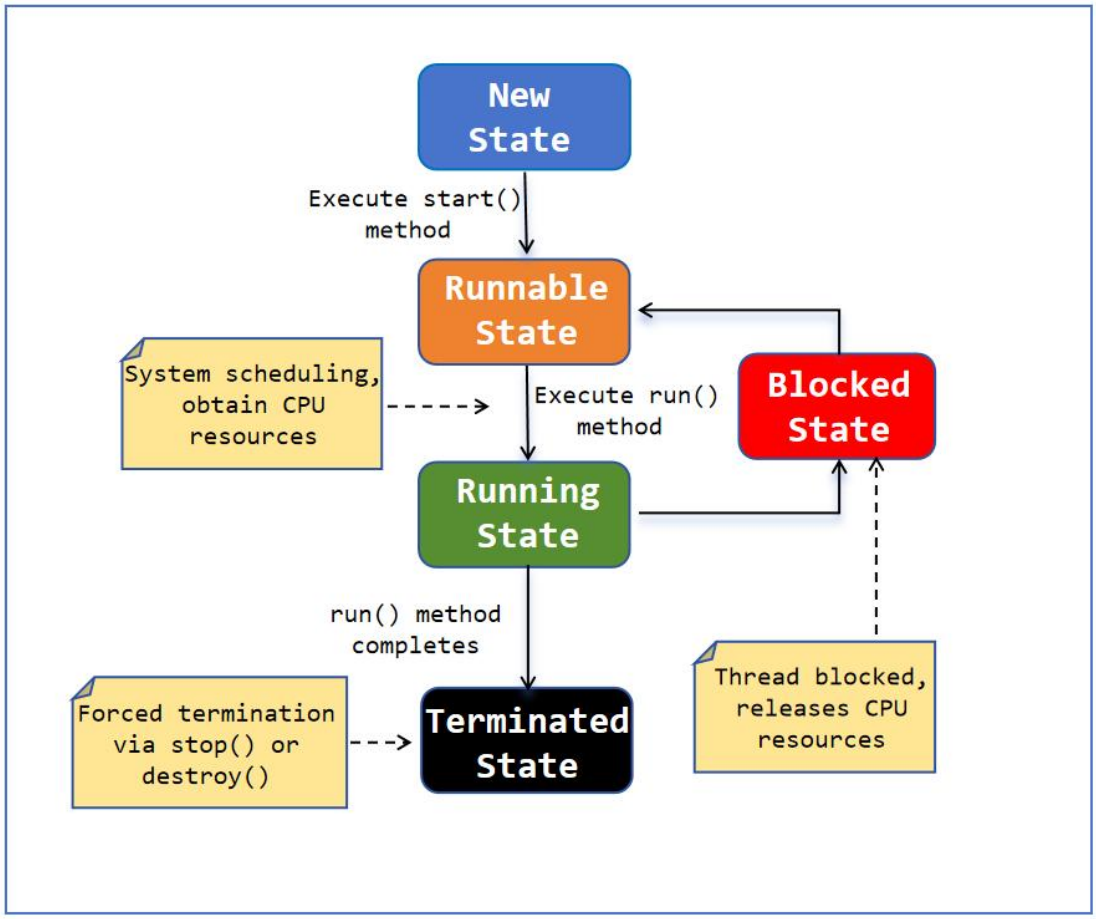
4.1 Multithreading in Java

In the AircraftWar game, features such as sound effect management and switching between firepower states must be implemented using multithreading. Java provides

built-in support for **multithreading**, allowing multiple threads of execution to run concurrently within a single program. A thread is a lightweight subprocess — the smallest unit of execution that can be scheduled by the operating system. In Java, every program starts with at least one thread: the main thread. Additional threads can be created to perform time-consuming or independent tasks — such as playing audio, handling timers, or managing network I/O — without blocking the main application logic.

A thread is a dynamically executing process that also undergoes a lifecycle — from creation to termination.

The figure below illustrates the complete **lifecycle of a thread**.



- ✧ **New State:** The thread has been instantiated but not yet started.
- ✧ **Runnable State:** The thread is ready to run and is waiting for the CPU scheduler to assign it processing time.
- ✧ **Blocked/Waiting/Timed Waiting State:**
  - ✓ **Blocked State:** The thread is waiting to acquire a monitor lock.
  - ✓ **Waiting State:** The thread is waiting indefinitely for another thread to

perform a particular action.

- ✓ **Timed Waiting State:** The thread is waiting for a specified period of time.

✧ **Terminated State:** The thread has finished execution either normally or abnormally .

Once a thread reaches the Terminated state, it cannot be restarted.

4.2 Ways to Implement Multithreading in Java

Java provides multiple ways to create threads. The following section introduces two commonly used approaches: implementing the Runnable interface and extending the Thread class itself to achieve multithreading.

(1) Implementing the Runnable interface

The Runnable interface can be implemented in several ways: through a standalone (top-level) class, a static nested class, or an anonymous inner class. Because Runnable is a functional interface—containing only a single abstract method (run())—it can also be instantiated concisely using a lambda expression. This approach helps reduce boilerplate code associated with anonymous classes. Once a Runnable instance is created, it can be passed to a Thread constructor, and the thread can be started by invoking the start() method.

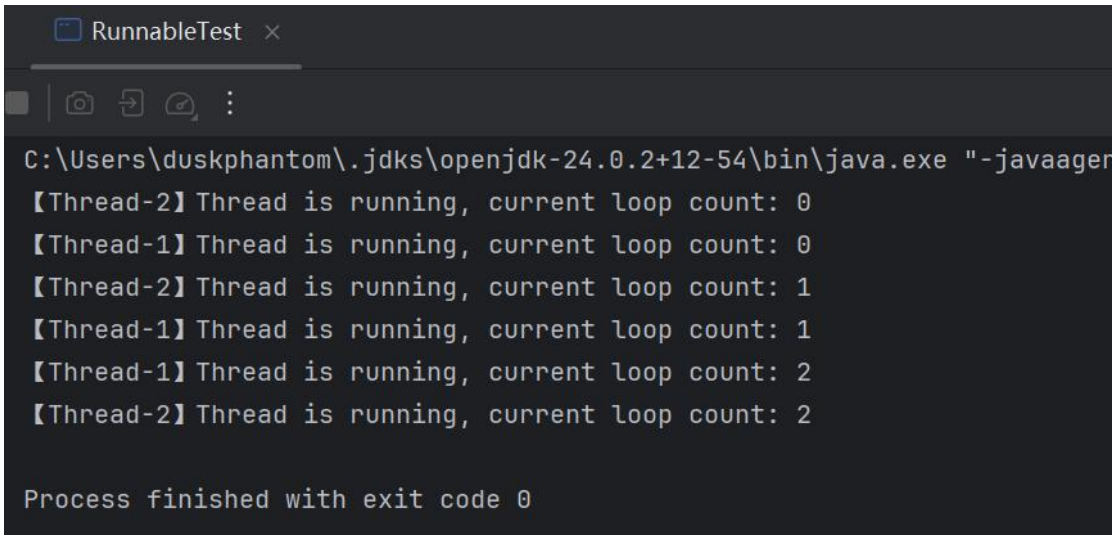
Code example of Runnable interface:

Define a Runnable task using a lambda expression. This task prints the current thread's name and loop index, then sleeps for 2 seconds in each iteration (simulating work).

Runnable.java

```
public class RunnableTest {
	public static void main(String[] args) {

		Runnable r = () -> {
			try {
				for (int i = 0; i < 3; i++) {
					System.out.println("【" + Thread.currentThread().getName() + "】" +
							"Thread is running, current loop count: " + i);
					Thread.sleep(2000);
				}
			} catch (InterruptedException e) {
				e.printStackTrace();
			}
		};
		// Create and start two threads that execute the same Runnable task.
		new Thread(r, "Thread-1").start();
		new Thread(r, "Thread-2").start();
	}
}
```

Program Output:



(2) Extending the Thread class

Create a subclass of java.lang.Thread and override the run() method directly. An instance of this subclass is itself a thread and can be started by invoking its start() method. While straightforward, this method is less flexible because Java does not support multiple inheritance—your class cannot extend another class if it already extends Thread.

Code example of extending the Tread class:

Define a custom thread class that extends Thread. Each instance runs its own copy of the task defined in run().

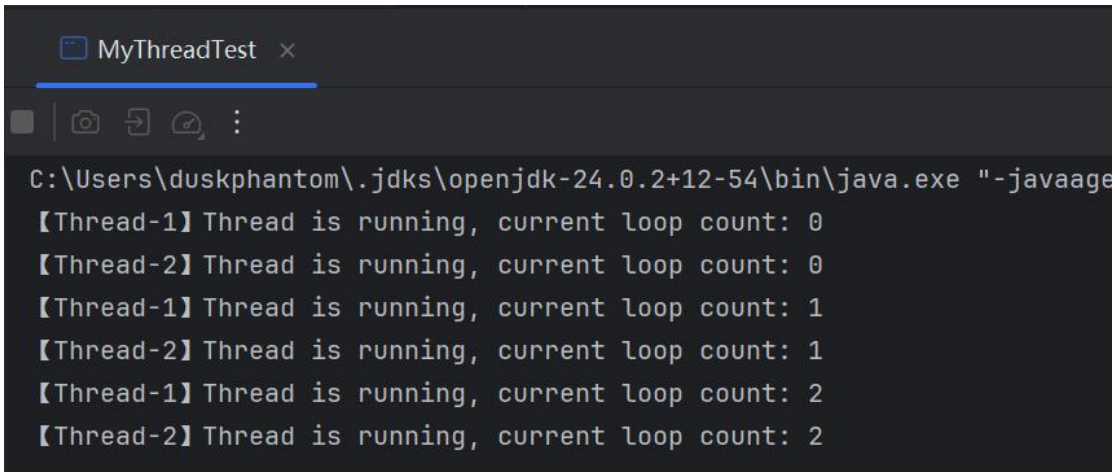
MyThread.java

```
public class MyThread extends Thread {
    // Constructor to set the thread name
    public MyThread(String name) {
        super(name); // Pass name to the parent Thread class
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < 3; i++) {
                System.out.println("【" + Thread.currentThread().getName() + "】" +
                        "Thread is running, current loop count: " + i);
                Thread.sleep(2000);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

MyThreadTest.java

```
public class MyThreadTest {
    public static void main(String[] args) {
        // Create and start two instances of the custom thread class
        new MyThread("Thread-1").start();
        new MyThread("Thread-2").start();
    }

}
```

Program Output:



4.3 Multithreaded Implementation of Firepower Power-ups

Use the Runnable interface to implement multithreaded timing control for firepower power-ups.

Tasks to be completed:

a. When a firepower is activated, the hero aircraft must immediately switch to an enhanced firing trajectory and **remain in this state for exactly 5 seconds** (You can useThread.sleep() method).

b. After the 5-second duration elapses, the aircraft should **automatically revert to its default straight-shot** trajectory.

c. You must implement this timed behavior using multithreading (**via the Runnable interface**) to ensure it does not block the main game loop.

4.4 Multithreaded Implementation of Audio Control

This experiment provides a MusicThread class that extends Thread and overrides the run() method to control audio playback. **It supports optional looping and can be stopped on demand.**

Note: Please focus on the parts of the code marked in red. The audio playback logic is provided for reuse and does not need to be understood in detail at this stage.

MusicThread.java

```
package edu.hitsz.music;

import javax.sound.sampled.*;
import javax.sound.sampled.DataLine.Info;
import java.io.*;

//A thread class for playing audio files using Java Sound API.
public class MusicThread extends Thread {
    // Flag indicating whether the audio should loop continuously
    private boolean isRepeat = false;

    // Flag to signal that playback should stop immediately
    private boolean isStop = false;

    // Path to the audio file to be played
    private final String filename;

    // The audio format of the loaded fil
    private AudioFormat audioFormat;

    // Raw audio data as a byte array, loaded from the file
    private byte[] samples;

    /**
     * Constructor: initializes the music player with a given file and repeat setting.
     *
     * @param filename    Path to the audio file (e.g., "music.wav")
     * @param isRepeat    Whether to loop the audio after it finishes
     */
    public MusicThread(String filename, boolean isRepeat)
    {
        this.filename=filename;
        this.isRepeat = isRepeat;
        reverseMusic();
    }
```

```
    //Loads the audio file and extracts its format and raw sample data.
    //This method reads the file once at initialization to avoid repeated I/O during playback.
    public void reverseMusic() {
        ...
    }

    //Reads all audio samples from the given AudioInputStream into a byte array.
    public byte[] getSamples(AudioInputStream stream) {
        ...
    }

    //Plays audio from the given InputStream using a SourceDataLine.
    public void play(InputStream source) {
      ...
    }

    /**
     * Main thread execution method.
     * Plays the audio repeatedly if isRepeat is true.
     */
    @Override
    public void run() {
        do {
            // Create a new input stream from the pre-loaded samples
            InputStream stream = new ByteArrayInputStream(samples);
            play(stream);
        } while (isRepeat);
    }

    //Signals the thread to stop playback immediately.
    public void stopPlay() {
        this.isStop = true;
    }
}
```

How to Use the MusicThread Class to Control Audio Playback?

The provided MusicThread class extends Thread and encapsulates audio playback logic. Here's how to use it in your game:

MusiceTest .java

```
public class MusicTest {
    public static void main(String[] args) throws InterruptedException {
        // Create and start a looping background music thread.
        MusicThread bgm = new MusicThread("src/videos/bgm.wav",true);

        // Call start() to begin playing the audio in a separate thread
        bgm.start();
```

```
        // Let the background music play for 5 seconds.
        Thread.sleep(5000);

        //To stop the music (e.g., when the game ends )
        bgm.stopPlay();

        // Play a one-time sound effect (e.g., bullet hit).
        new MusicThread("src/videos/bullet_hit.wav",false).start();
    }
}
```

To better manage audio playback, **this experiment also provides a MusicController class**. You are required to complete the sections marked with **TODO** in this class.

Guidelines:

- Background music (BGM) and boss music should loop continuously and must be controlled via dedicated thread references so they can be stopped later.
- One-time sound effects (e.g supply pickup) should be played once and do not need to be stored as instance variables.

Tasks to be completed:

1. Refer to the implemented methods startBgmMusic() and stopBgmMusic().
2. Complete all methods marked with "TODO".
3. Remember:

- Looping sounds: create and store a MusicThread (like bgmThread).
- One-shot sounds: create a new MusicThread(..., false) and call .start() immediately.

StrategyPatternDemo.java

```
package edu.hitsz.music;
/**
 * Audio controller class for managing background music and sound effects in a game.
 */
public class MusicController {

    // File paths for audio resources
    public static String bgmMusic = "src/videos/bgm.wav";
    public static String bossMusic = "src/videos/bgm_boss.wav";

    // Keep references to looping music threads so we can stop them later
    private MusicThread bgmThread;
    private MusicThread bossThread;


    // Constructor: initializes the background music thread (set to loop).
    public MusicController() {
        bgmThread = new MusicThread(bgmMusic, true);
    }
```

```
//Starts playing the background music (loops indefinitely).
public void startBgmMusic() {
	bgmThread.start();
}

//Stops the background music.
public void stopBgmMusic() {
	bgmThread.stopPlay();
}

/**
* Starts playing the boss music (should loop).
* TODO: Implement this method by creating and starting a looping MusicThread for
* bossMusic.
*Store it in the bossThread field so it can be stopped later.
*/
public void startBossMusic() {
	// TODO
}

/**
* Stops the boss music.
* TODO: Stop the bossThread if it is not null.
*/
public void stopBossMusic() {
	// TODO
}

/**
* Plays the bullet hit sound effect (one-time playback).
* TODO: Create a new non-looping MusicThread for bulletHitMusic and start it
* immediately.
*/
public void startBulletHitMusic() {
	// TODO
}

/**
* Plays the supply pickup sound effect (one-time playback).
* TODO: Create a new non-looping MusicThread for supplyMusic and start it
* immediately.
*/
public void startSupplyMusic() {
	// TODO
}


/**
* Plays the game over sound effect (one-time playback).
* TODO: Create a new non-looping MusicThread for gameOverMusic and start it
```

```
     *immediately.
     */
     public void startGameOverMusic() {
          // TODO
     }
}
```

Once you have completed the implementation of the MusicController class, you can use it to refactor your game code and add sound effects.

Tasks to be completed:

a. Implement continuous looping of **background music** during normal gameplay, and ensure it stops immediately when the game ends.

b. Play appropriate sound effects at the following key moments:

- ✓ When a **bullet hits** an enemy aircraft
- ✓ When a **power-up supply takes effect**
- ✓ When the **game ends**

c. Trigger dedicated **Boss background music** as soon as the Boss enemy appears.

d. Stop the Boss music either when the Boss is destroyed or when the game ends—whichever occurs first.

e. Use the **MusicController class** (which you have implemented) to manage all audio playback, ensuring thread-safe and non-blocking operation.

The audio files are located in the src/videos directory. The files you will need are as follows:

- ✓ bgm.wav (game background music)
- ✓ bgm_boss.wav (boss background music)
- ✓ bullet_hit.wav (when a bullet hits an enemy aircraft)
- ✓ game_over.wav (when the game ends)
- ✓ get_supply.wav (when a power-up supply takes effect)

5. Assignment Submission

Submission requirements include:

Only need to submit the Project Code (the entire project folder ”AircraftWar-base”)
Compress the entire folder into a single archive file (.zip) for submission.

Deadline:

Submit your work to the **HITsz Grader** assignment submission platform **within one week** after the lab session. The exact due date will be specified on the platform.

Login URL: https://grader.cs-lab.top/

The default username and password are your student ID.

Note: After uploading, please download your submission to verify that it was uploaded successfully.